

Calcolatori Elettronici

25 gennaio 2023

Teoria

Esercizio 1 (12 punti): Data una sequenza infinita di caratteri in ingresso appartenenti all'alfabeto "i, n, o, p", progettare la rete sequenziale che riconosca le occorrenze della sequenza "pino", ipotizzando che l'alfabeto di uscita sia "sequenza riconosciuta, sequenza non riconosciuta". Realizzare il circuito equivalente dell'equazione minimizzata di eccitazione di un singolo flip flop di stato.

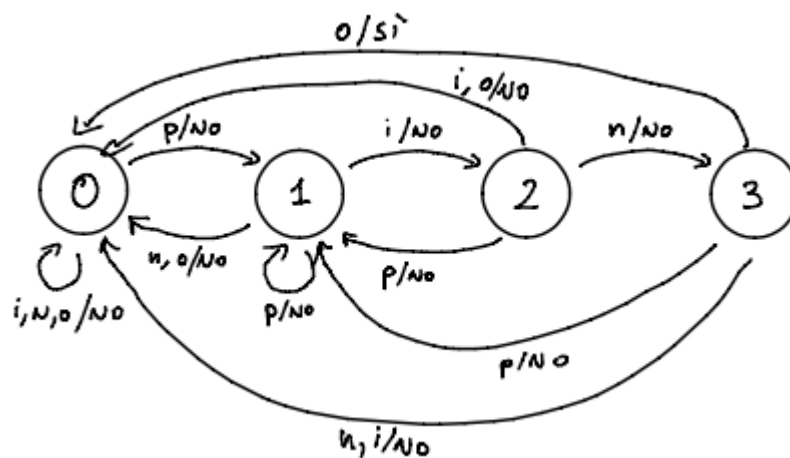
Esercizio 2 (12 punti): Si considerino i numeri $A=3X$ e $B=-1Y$, dove X e Y sono rispettivamente il penultimo e l'ultimo numero della propria matricola. Convertire entrambi i numeri in complemento a due a 8 bit e calcolare $A+B$ mostrando tutti i passaggi. Convertire il risultato in virgola mobile, utilizzando 6 bit di esponente e 10 bit di mantissa.

Esercizio 3 (6 punti): Discutere vantaggi e svantaggi della memoria cache completamente associativa.

Soluzioni

Esercizio 1

La sequenza è *infinita* e si chiede di riconoscere le *occorrenze*, pertanto si può organizzare la macchina come segue:



Codifica dell'alfabeto in input:

	x_1	x_0
i	0	0
n	0	1
o	1	0
p	1	1

Codifica dell'alfabeto in output:

	z
sequenza riconosciuta	1
sequenza non riconosciuta	0

Tabella degli stati e transizioni:

y_1	y_0	x_1	x_0	y'_1	y'_0	z
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	0	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	1	1	0	1	0
1	0	0	0	0	0	0
1	0	0	1	1	1	0
1	0	1	0	0	0	0
1	0	1	1	0	1	0
1	1	0	0	0	0	0
1	1	0	1	0	0	0
1	1	1	0	0	0	1
1	1	1	1	0	1	0

Scegliendo l'equazione di eccitazione del flip flop che memorizza il valore y_1 , ci si accorge che essa è composta da due mintermini non adiacenti. Pertanto, la funzione minima in forma normale disgiuntiva è direttamente presa dai mintermini:

$$y'_1 = \bar{y}_1 y_0 \bar{x}_1 \bar{x}_0 + y_1 \bar{y}_0 \bar{x}_1 x_0$$

che porta al circuito:

Esercizio 2

Assumendo $x=9$ e $y=0$ si ha:

- $A=39 \Rightarrow (39)_{10} = (00100111)_2$

infatti:

valore	/ 2	resto
39	19	1
19	9	1
9	4	1
4	2	0
2	1	0
1	0	1

- $B=-10 \Rightarrow (10)_{10} = (00001010)_2 \Rightarrow (-10)_{10} = (11101110)_2$

La somma è quindi:

```
1  0 0 1 0 0 1 1 1 +
2  1 1 1 1 0 1 1 0 =
3  -----
4  0 0 0 1 1 1 0 1
```

Per la rappresentazione in virgola mobile si ottiene:

$$11101 = 1.1101 \cdot 2^4$$

Utilizzando 6 bit di esponente, si può utilizzare un eccesso a $2^5 = 32$ per l'esponente, pertanto:

$$e = 32 + 4 = (36)_{10} = (100100)_2$$

Poiché il risultato è positivo, la codifica in virgola mobile sarà:

```
1  0 | 100100 | 1101000000
```

Progetto

Un processore z64 gestisce un passaggio a livello automatizzato a binario singolo, composto da un dispositivo SBARRA e da due dispositivi B0A. I treni transitano sul binario in *entrambe le direzioni*.

Il dispositivo B0A informa il processore dell'avvicinamento di un treno da una delle due direzioni in maniera *asincrona*. Quando il processore viene allertato, esso programma il dispositivo SBARRA in maniera *sincrona* per abbassarsi.

Quando il treno è transitato, il secondo dispositivo **BOA** informa lo z64 del fatto che il treno ha raggiunto una distanza di sicurezza sufficiente per alzare la sbarra. Il processore, quindi, programmerà (sempre in maniera *sincrona*) il dispositivo **SBARRA** per alzarsi.

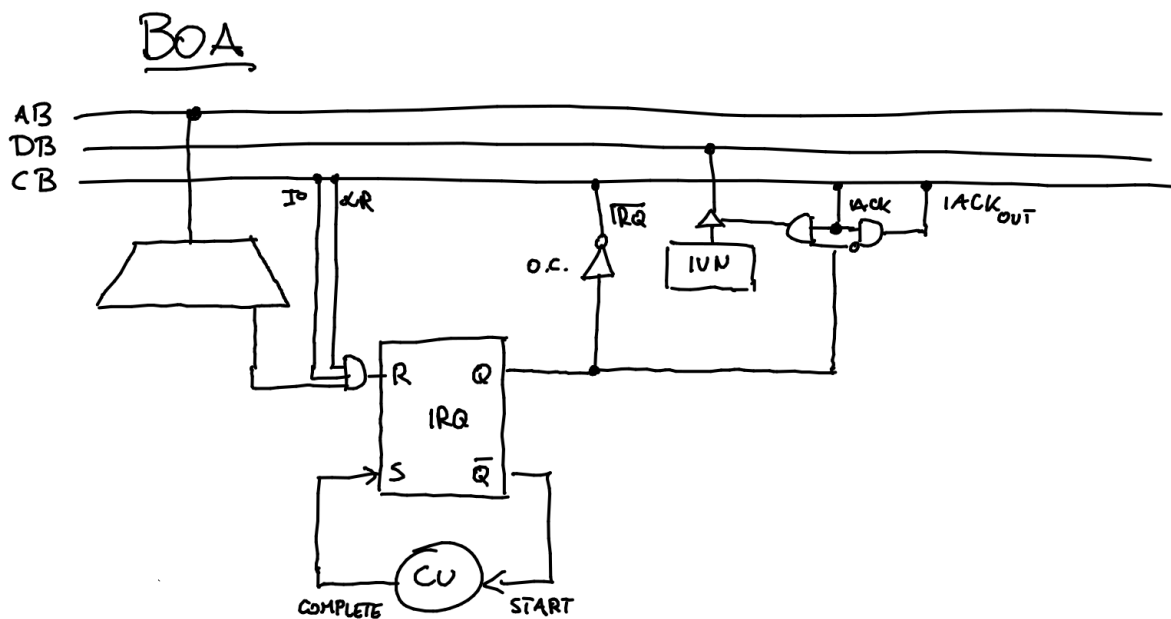
Nel caso in cui un treno transiti in direzione opposta, il processore z64 verrà informato dai due dispositivi **BOA** in ordine inverso. Il funzionamento di **SBARRA** dovrà però essere il medesimo (prima si abbassa, poi si alza).

Realizzare:

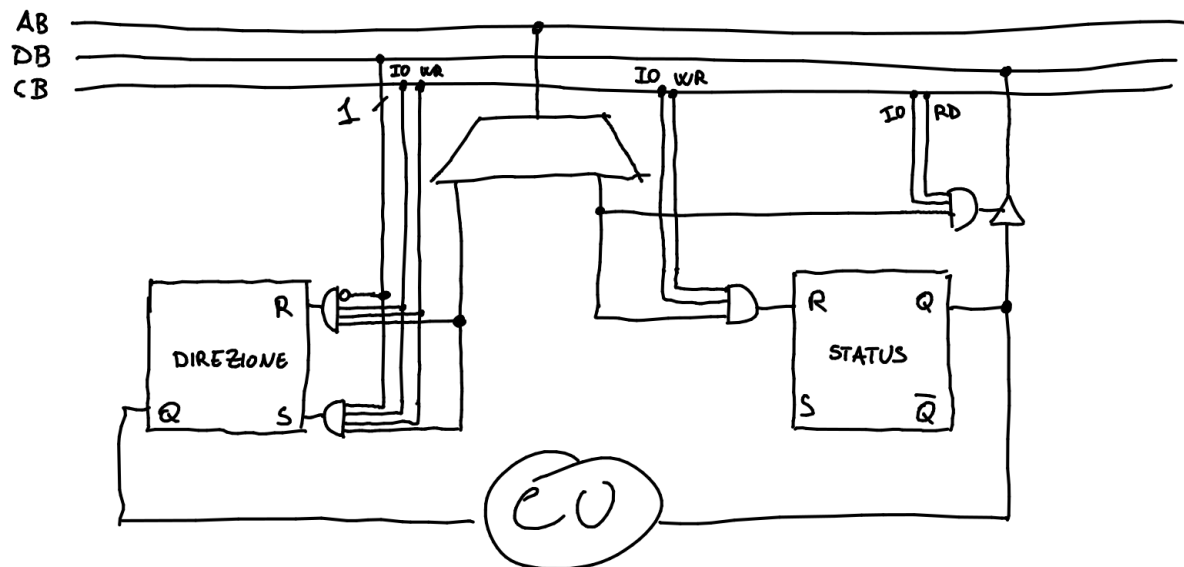
- Le interfacce dei dispositivi **BOA** e **SBARRA**
- Tutto codice necessario al funzionamento del passaggio a livello.

Soluzione

Interfacce delle periferiche:



In questa interfaccia si suppone che la boa possa ricominciare a mandare una richiesta di interruzione quando la precedente è stata servita.



Il flip flop di direzione permette di dire alla CU se deve alzare o abbassare la sbarra. Non è strettamente necessario, poiché si potrebbe realizzare la CU in maniera tale da far transitare la sbarra dallo stato "alzato" allo stato abbassato. Tuttavia, in questo caso, occorre fare una supposizione sullo stato iniziale della sbarra (ad esempio: all'inizio la sbarra è alzata) che non si è ritenuto di voler fare in questa soluzione.

Il codice può essere il seguente:

```

1  .org 0x800
2  .data
3      operazione: .byte 0 # La prossima operazione per SBARRA
4      .equ BOA_SX_IRQ, 0x1
5      .equ BOA_DX_IRQ, 0x2
6      .equ SBARRA_DIR, 0x3
7      .equ SBARRA_STATUS, 0x4
8  .text
9      main:
10     sti
11     .loop:
12     hlt
13     jmp loop
14
15     .driver 1 # BOA di sinistra
16     call commuta_sbarra
17     outb %al, $BOA_IRQ
18     iret
19     .driver 2 # BOA di destra
20     call commuta_sbarra
21     outb %al, $BOA_IRQ
22     iret
23
24     commuta_sbarra:
25     pushb %al
26     movb operazione, %al
27     outb %al, $SBARRA_DIR
28     outb %al, $SBARRA_STATUS
29     .bw:
30     inb $SBARRA_STATUS, %al
31     btb $0, %al
32     jnc .bw
33     addb $1, operazione
34     popb %al
35     ret

```

La funzione `commuta_sbarra` fa effettivamente commutare la sbarra poiché l'interfaccia prende soltanto il bit meno significativo dal data bus. L'istruzione `add` alla riga 33 fa effettivamente invertire il bit meno significativo, anche prendendo in considerazione l'overflow della variabile. Un'alternativa senza questo hack richiede di sostituire l'istruzione alla riga 33 con `xorb $1, operazione`.

Prova di laboratorio

Si vuole realizzare un programma in grado di trasformare un file di testo, in formato *comma-separated values* (CSV), in formato binario e viceversa.

In particolare, il programma accetta come argomento a linea di comando un selettore di operazione ed un path ad un file. Ad esempio, se il programma viene invocato come:

```
1 | ./convert --to-binary input.csv
```

esso effettuerà la conversione da formato CSV al formato binario, creando un file `converted.bin`. Viceversa, se invocato come:

```
1 | ./convert --to-csv input.bin
```

effettuerà la conversione da formato binario a formato CSV, creando il file `converted.csv`. I file CSV in input mantengono, in ciascuna linea, dei record nel seguente formato (senza spazi prima e dopo le virgole):

```
1 | nome,cognome,età,email
```

e la rispettiva conversione in formato binario richiede di popolare una sequenza di `struct` nel formato:

```
1 | struct persona {  
2 |     char nome[32];  
3 |     char cognome[32];  
4 |     unsigned eta;  
5 |     char email[64];  
6 | };
```

Il programma realizzato deve implementare una funzione `convert` con la seguente firma:

```
1 | bool convert(enum operation op, FILE *f, bool (*conv)(FILE *f, char *outname));
```

dove `conv` è un puntatore ad una funzione che effettua la conversione da CSV a binario o da binario a CSV, `f` è l'handle del file che si vuole processare, `outname` è il nome del file in cui si vuole memorizzare l'output, `op` è un descrittore di operazione nel formato:

```
1 | enum operation {  
2 |     TO_CSV,  
3 |     TO_BIN  
4 | };
```

Il valore di ritorno di `convert` determina se c'è stato un errore durante la manipolazione delle informazioni. In questo caso, il programma deve terminare con un valore di ritorno diverso da zero.

Per semplicità si può supporre che i file passati in input siano ben formati e non contengano errori. Vengono forniti due file di esempio per testare le funzionalità di conversione.

Soluzione

```
1  #include <stdbool.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  #define LINE_LEN 1024
7
8  enum operation { TO_CSV, TO_BIN };
9
10 enum fields {
11     NOME,
12     COGNOME,
13     ETA,
14     EMAIL,
15 };
16
17 struct persona {
18     char nome[32];
19     char cognome[32];
20     unsigned eta;
21     char email[64];
22 };
23
24 static void converti_persona(struct persona *p, char *line)
25 {
26     enum fields current = NOME;
27     char *prev = line, *c = line;
28
29     while(*c) {
30         if(*c == ',' || *c == '\n') {
31             *c = 0;
32
33             switch(current) {
34                 case NOME:
35                     strcpy(p->nome, prev);
36                     break;
37                 case COGNOME:
38                     strcpy(p->cognome, prev);
39                     break;
40                 case ETA:
41                     // Il file è assunto non avere errori
42                     p->eta = atoi(prev);
43                     break;
44                 case EMAIL:
45                     strcpy(p->email, prev);
46                     break;
47                 default:
48                     fprintf(stderr, "Unexpected case at %s:%d", __FILE__, __LINE__);
49                     abort();
50             }
51
52             current++;
53             prev = c + 1;
54         }
55         c++;
56     }
57 }
58
59 static bool to_csv(FILE *f, char *outname)
```

```

60 {
61     FILE *out = fopen(outname, "wb");
62     struct persona p;
63
64     if(out == NULL)
65         return false;
66
67     for(;;) {
68         size_t n = fread(&p, sizeof(p), 1, f);
69
70         if(n == 0) {
71             if(ferror(f))
72                 return false;
73             break;
74         }
75
76         fprintf(out, "%s,%s,%d,%s\n", p.nome, p.cognome, p.eta, p.email);
77     }
78
79     return true;
80 }
81
82 static bool to_bin(FILE *f, char *outname)
83 {
84     char line[LINE_LEN];
85     struct persona p;
86
87     FILE *out = fopen(outname, "wb");
88     if(out == NULL)
89         return false;
90
91     while(fgets(line, sizeof(line), f) != NULL) {
92         converti_persona(&p, line);
93         if(fwrite(&p, sizeof(struct persona), 1, out) != 1)
94             return false;
95     }
96
97     return true;
98 }
99
100 bool convert(enum operation op, FILE *f, bool (*conv)(FILE *f, char *outname))
101 {
102     char *outname;
103
104     if(op == TO_BIN)
105         outname = "converted.bin";
106     else if(op == TO_CSV)
107         outname = "converted.csv";
108     else
109         return false;
110
111     return conv(f, outname);
112 }
113
114
115 int main(int argc, char **argv)
116 {
117     bool ret;
118
119     if(argc != 3) {
120         fprintf(stderr, "Usage: %s [--to-binary/--to-csv] <path-to-file>", argv[0]);
121         exit(EXIT_FAILURE);
122     }
123
124     FILE *f = fopen(argv[2], "rb");
125     if(f == NULL) {

```



```
126         fprintf(stderr, "Unable to open file: %s", argv[2]);
127         exit(EXIT_FAILURE);
128     }
129
130     if(strcmp(argv[1], "--to-binary") == 0) {
131         ret = convert(TO_BIN, f, to_bin);
132     } else if(strcmp(argv[1], "--to-csv") == 0) {
133         ret = convert(TO_CSV, f, to_csv);
134     } else {
135         fprintf(stderr, "Unknown operation: %s", argv[1]);
136         exit(EXIT_FAILURE);
137     }
138
139     return ret != true;
140 }
141
```

